Building a Custom MCP Server — Healthcare Patient Records

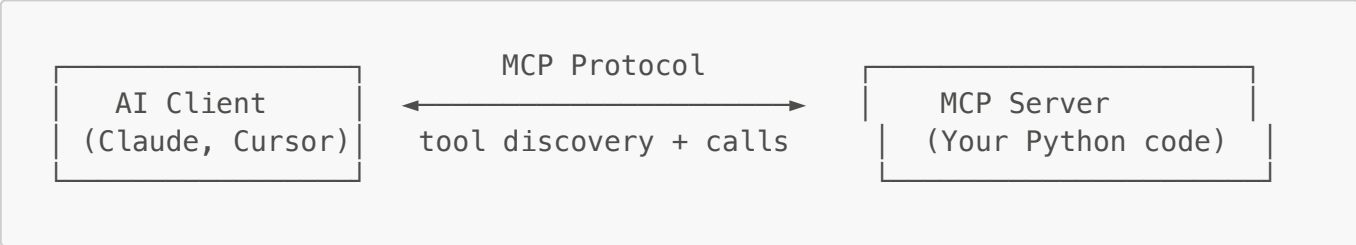
A Step-by-Step Technical Guide

1. What Is MCP (Model Context Protocol)?

When you use an AI assistant like Claude, it is very good at understanding language and reasoning — but it has no built-in access to your data, your databases, or your systems. It only knows what you tell it in the conversation.

MCP — Model Context Protocol — is an open standard developed by Anthropic that solves this problem. It defines a structured way for AI models to discover and call external tools, just like a web browser can call external APIs.

Think of MCP as a **universal plugin system for AI**. Instead of every AI vendor inventing their own format for tool calling, MCP provides one standard that works across different AI clients (Claude Desktop, Cursor, Continue, custom agents).



Key Concepts at a Glance

Term	Meaning
MCP Server	Your code that exposes tools, resources, or prompts
MCP Client	The AI application (Claude Desktop, an agent) that uses those tools
Tool	A callable function with a name, description, and typed parameters
Transport	How client and server communicate — Stdio (local) or Streamable HTTP (remote)
Streamable HTTP	The modern MCP transport — a single HTTP endpoint (`/mcp`) that handles both requests and streaming responses

2. Why Build a Custom MCP Server?

Out-of-the-box AI assistants can browse the web, run code, and read files. But in enterprise and healthcare settings, you need the AI to:

- Query a **proprietary database** (patient records, EHR systems)
- Apply **domain-specific logic** (clinical risk scoring algorithms)

- Enforce **security guardrails** (block write operations, audit access)
- Return **structured, typed data** the AI can reason about reliably

A custom MCP server lets you expose exactly these capabilities — as clean, well-described tools — without giving the AI unstructured access to your systems.

For this project, we built four tools around a healthcare SQLite database:

- Risk scoring from diagnoses, medications, and lab results
- Readmission risk analysis from visit history
- Schema introspection so the AI understands the database structure
- A safe SQL interface for ad-hoc queries

---

3. Transport Options: Stdio vs Streamable HTTP

The MCP spec supports two transport mechanisms:

Stdio (Local)

- Server and client run on the same machine
- Communication happens via standard input/output (stdin/stdout)
- Best for local tools like file readers, code runners
- Used by Claude Desktop for locally installed MCP servers

Streamable HTTP (Remote)

- Server runs as a network service (any host/port)
- Client sends HTTP POST requests to a single `/mcp` endpoint
- Responses can be streamed or returned as plain JSON
- Supersedes the older SSE transport — simpler, stateless-friendly, and broadly compatible
- Supports multiple concurrent clients
- Best for shared infrastructure, cloud deployments, team-wide tools

We chose Streamable HTTP because:

1. It allows the server to run independently and be shared across multiple AI clients
2. It is production-ready — you can deploy it to any cloud or internal server
3. It is the current MCP standard, replacing the older SSE-based transport

```
AI Client
   │
   │  HTTP POST /mcp
   ▼
MCP Server (http://0.0.0.0:8000/mcp)
   │
   │  Reads from patients.db
   ▼
SQLite Database
```

4. Project Architecture

```
healthcare-mcp-server/
├── patients.db          ← SQLite database (5 tables, 10 patients)
├── seed_data.py         ← One-time script to populate sample data
├── db.py                ← Shared database connection utility
├── tools/
│   ├── risk_summary.py  ← Tool 1A: Patient risk scoring
│   ├── readmission.py   ← Tool 1B: Readmission risk
│   ├── schema_tool.py   ← Tool 2A: Schema introspection
│   └── query_tool.py    ← Tool 2B: Safe SQL execution
├── server.py            ← MCP server entry point
└── requirements.txt     ← mcp[cli], uvicorn, starlette
```

Design principle: Every tool function is independently testable. You can run `python tools/risk_summary.py` without the MCP server running at all. The server is just the final layer that wraps these functions and exposes them over the network.

5. Database Design

We used SQLite because it requires no server setup and ships with Python. The database has five tables:

`patients` — Core Demographics

```
patient_id, name, age, gender, blood_type, contact_number, created_at
```

`visits` — Hospital Visit Records

```
visit_id, patient_id, visit_date, discharge_date, visit_type, department,
attending_doctor
```

`visit_type` values: Emergency, Routine, Follow-up

`diagnoses` — Medical Diagnoses per Visit

```
diagnosis_id, patient_id, visit_id, diagnosis_code, diagnosis_name,
diagnosis_type, diagnosed_date
```

`diagnosis_type` values: `acute` (short-term) or `chronic` (ongoing)

`medications` — Prescriptions

```
medication_id, patient_id, medication_name, dosage, start_date, end_date,
is_active
```

`is_active = 1` means the medication is currently being taken.

`lab_results` — Lab Test Results

```
lab_id, patient_id, visit_id, test_name, result_value, unit,
reference_range, is_abnormal, test_date
```

`is_abnormal = 1` means the result is outside the reference range.

Entity Relationship Summary

```
patients (1) ——— (many) visits
patients (1) ——— (many) diagnoses
patients (1) ——— (many) medications
patients (1) ——— (many) lab_results
visits   (1) ——— (many) diagnoses
visits   (1) ——— (many) lab_results
```

---

6. The Four Tools — Implementation Details

Tool 1A: `get_patient_risk_summary`

Purpose: Produce a clinical risk profile for a patient.

Data sources queried:

1. `patients` — name, age, gender
2. `diagnoses` — count of distinct chronic conditions
3. `medications` — count of active prescriptions
4. `lab_results` — most recent result per test, flagged if abnormal

Scoring logic:

Signal	How it's counted
Chronic conditions	1 point each, capped at 3
Polypharmacy	1 point per every 2 meds beyond the first 2
Abnormal labs	1 point each (no cap)

Total Signals	Risk Tier

Total Signals	Risk Tier
0–2	LOW
3–4	MEDIUM
5+	HIGH

Key implementation detail: For lab results, we only consider the *most recent* result per test type. An old abnormal HbA1c that has since improved should not keep penalizing the patient.

Tool 1B: `get_readmission_insights`

Purpose: Identify patients at risk of being readmitted to the hospital.

Data sources queried:

1. `visits` — all visits ordered by date descending
2. `diagnoses` — grouped by name across visits to detect recurring conditions

Computed fields:

- Visit counts for 30, 90, and 180-day windows
- Days since last discharge (compared to today's date)
- List of diagnoses that appear in 2+ separate visits

Risk classification:

Condition	Risk Level
3+ visits in last 90 days OR discharged <30 days ago with recurring dx	HIGH
2 visits in 90 days OR any recurring diagnosis	MODERATE
Otherwise	LOW

A plain-English summary is generated automatically — this is what the AI returns to the user in natural language.

Tool 2A: `get_database_schema`

Purpose: Utility tool for AI agents to understand the database before querying.

Why this matters: When an AI agent wants to write a SQL query, it needs to know:

- Which tables exist
- What columns each table has and their types
- How many rows are in each table (scale of data)

This tool calls SQLite's `PRAGMA table_info()` for each table and returns a clean nested dictionary. The AI agent should always call this tool first before `query_database`.

Tool 2B: `query_patient_database`

Purpose: Allow the AI to run ad-hoc SQL queries safely.

Safety guardrail: The function checks if the query starts with `SELECT` (case-insensitive). Anything else — `INSERT`, `UPDATE`, `DELETE`, `DROP` — is rejected immediately with a clear error message. This means the AI can explore data but cannot modify or destroy it.

Return format:

```
{
    "status":    "success",
    "sql":       "SELECT ...",
    "row_count": 5,
    "results":   [{"name": "...", "age": 72}, ...]
}
```

Results are returned as a list of dictionaries, which the AI can read naturally.

---

7. Building the MCP Server with FastMCP

The MCP Python SDK provides a high-level class called `FastMCP` that handles all the protocol boilerplate. Here is the complete pattern:

```
from mcp.server.fastmcp import FastMCP
import uvicorn

# 1. Create the server instance
mcp = FastMCP(name="healthcare-mcp-server")

# 2. Register a tool with the decorator
@mcp.tool()
def patient_risk_summary(patient_id: int) -> dict:
    """
    Compute a risk profile for a patient.
    ...
    """
    return get_patient_risk_summary(patient_id)

# 3. Build the Streamable HTTP ASGI app and run it
app = mcp.streamable_http_app()  # exposes a single /mcp endpoint

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

What FastMCP does automatically:

- Reads the function's **type hints** (`patient_id: int`) and generates a JSON Schema for tool parameters
- Reads the **docstring** to create the tool's description that the AI sees
- Handles the MCP handshake, tool listing (`tools/list`), and tool calling (`tools/call`)
- Manages the Streamable HTTP transport lifecycle via a standard ASGI app

This is why type hints and docstrings are not optional — they are the mechanism by which FastMCP auto-generates the tool schema that the AI client uses to discover and call your tools correctly.

---

8. Step-by-Step Build Order

Here is exactly how we built the project, and why in this order:

Step 1 — Project structure and `requirements.txt` Set up the folder layout and install dependencies before writing any code.

Step 2 — `db.py` Build the database connection utility first. Every other module depends on it. Using `sqlite3.Row` as the row factory means all query results can be accessed by column name, making the code much more readable.

Step 3 — `seed_data.py` Populate the database with realistic data before building tools. Tools cannot be tested without data. We included:

- High-risk patients with multiple chronic conditions and abnormal labs (patients 1, 3, 4, 7)
- Moderate-risk patients with one or two chronic conditions (patients 2, 5, 6, 9, 10)
- A low-risk healthy baseline patient (patient 8, Charles Brown)

Step 4 — Tool files (independently) Build each tool as a standalone Python file with a `__main__` block. Test each with `python tools/risk_summary.py` before moving to the next. This separation means bugs in one tool don't affect others.

Step 5 — `server.py` Build the MCP server last, after all tools are verified. The server is just a thin registration layer — it imports the tool functions and wraps them in `@mcp.tool()` decorators. If a tool doesn't work in isolation, it won't work through MCP either.

Step 6 — Verify Start the server and confirm tools are listed correctly.

---

9. Running and Testing the Server

Start the server

```
python server.py
# Output: Starting Healthcare MCP Server on http://0.0.0.0:8000/mcp
```

Verify tools are listed (using MCP Inspector or CLI)

```
npx @modelcontextprotocol/inspector http://localhost:8000/mcp
```

Expected output — four tools listed:

```
patient_risk_summary    Compute a risk profile for a patient.
readmission_insights    Analyze a patient's visit history...
database_schema         Retrieve the schema of the patients.db...
query_database          Execute a read-only SQL SELECT query...
```

Connect Claude Desktop

Add this to `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "healthcare": {
      "url": "http://localhost:8000/mcp"
    }
  }
}
```

Then in Claude Desktop you can ask:

- *"What is the risk profile for patient 4?"*
- *"Which patients have HIGH readmission risk?"*
- *"Show me all patients with HbA1c above 9%"*

Claude will automatically discover and call the appropriate tools.

---

10. What's Next — Extending This Server

This project is a foundation. Here is how you could extend it for real-world use:

Add authentication

The current server has no authentication. In production, add an API key check or OAuth token validation before serving any tool calls.

Add more tools

Using the same pattern — a standalone Python function + `@mcp.tool()` — you can add:

- `get_medication_interactions` — check for drug-drug interactions
- `get_department_summary` — aggregate stats across all patients in a department
- `flag_critical_labs` — proactively alert on life-threatening lab values

Replace SQLite with PostgreSQL or an EHR API

The `db.py` module is the only place database connection logic lives. Swapping SQLite for a PostgreSQL connection (using `psycopg2`) or a REST API client requires changes in only one file.

Deploy to the cloud

Since the server uses Streamable HTTP over standard HTTP/HTTPS, you can deploy it to any cloud provider (AWS, GCP, Azure) behind a load balancer. Multiple AI clients can connect simultaneously.

Add resource and prompt primitives

MCP supports three primitives: **Tools** (what we built), **Resources** (read-only data like files or database records exposed as URIs), and **Prompts** (reusable prompt templates). Adding resources would let the AI browse patient records by ID without writing SQL.

This server was built for educational purposes. In a real healthcare deployment, all patient data must be handled in compliance with HIPAA and other applicable regulations.